DIGITAL DESIGN · PROJECT REVIEW



Design an 8-to-3 Priority Encoder

When several requests arrive, the highest-priority signal wins.

Combinational logic · specification, equations, verification, and applications

Why priority encoding is needed

A normal encoder assumes one active input. Real systems often receive several requests at once.

NORMAL ENCODER

One active input

Each input maps cleanly to one binary code. If two inputs rise together, the output is ambiguous.

Assumption: mutual exclusivity

PRIORITY ENCODER

Highest active input

Multiple active inputs are resolved deterministically. Here, **Y7** has the highest priority and **Y0** the lowest.

Rule: the highest asserted request wins

Inputs, outputs, and validity

Define the interface before deriving the logic.

INPUTS

Y7 … Y0

Eight active-high request inputs, ordered from highest to lowest priority.

OUTPUTS

A2 A1 A0

Three binary outputs identify the selected input in ordinary binary form.

VALID

V

V = 1 whenever at least one request is active. This separates “no request” from a valid Y0 selection, which also encodes as 000.

Signal contract: 8 request lines → 2-bit selection + 1 validity flag

Priority truth table

The highest asserted input wins; lower inputs are don't-care. V=1 when any input is active.

Highest active input	A2	A1	A0	V
None	0	0	0	0
Y0	0	0	0	**1**
Y1	0	0	1	**1**
Y2	0	1	0	**1**
Y3	0	1	1	**1**
Y4	1	0	0	**1**
Y5	1	0	1	**1**
Y6	1	1	0	**1**
Y7	1	1	1	**1**

Boolean equations and gate-level design

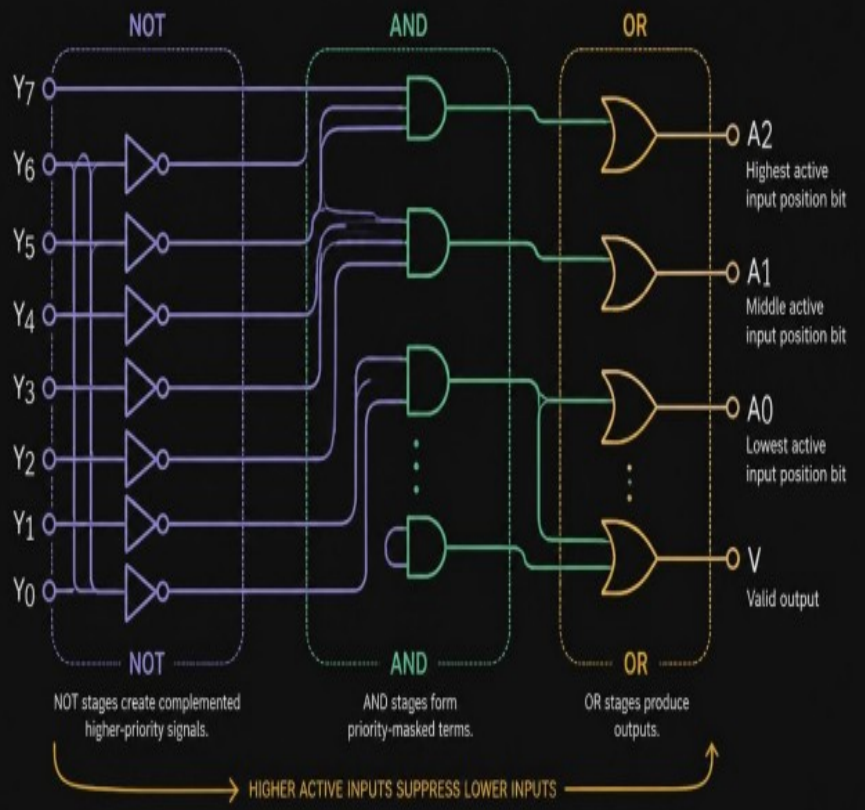
OUTPUT EQUATIONS

$$A_2 = Y_7 + Y_6 + Y_5 + Y_4$$

$$A_1 = Y_7 + Y_6 + Y_3 \cdot \bar{Y}_4 \cdot \bar{Y}_5 \cdot \bar{Y}_6$$

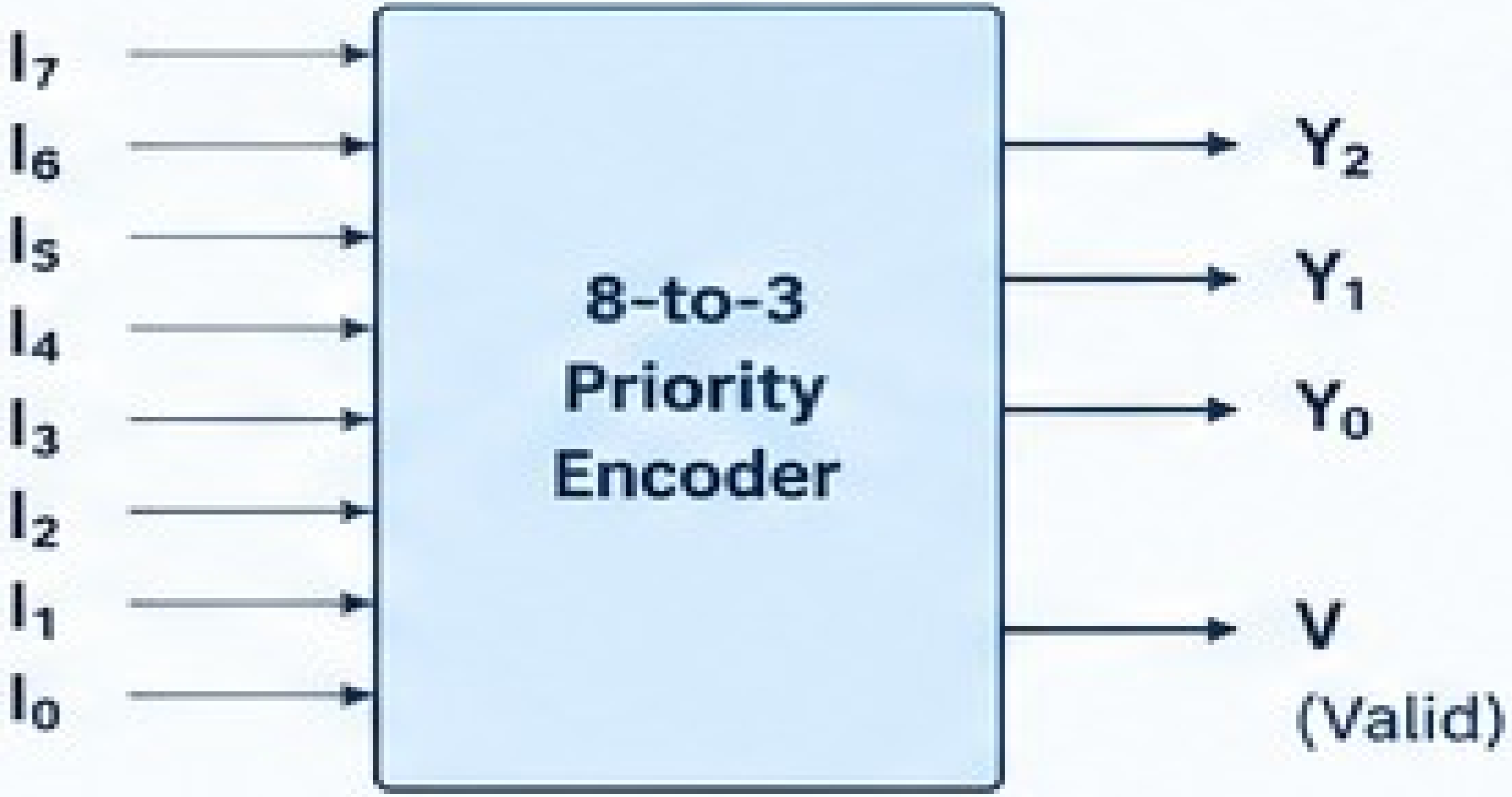
$$A_0 = Y_7 + Y_5 \cdot \bar{Y}_6 \cdot \bar{Y}_7 + Y_3 \cdot \bar{Y}_4 \cdot \bar{Y}_5 \cdot \bar{Y}_6 + Y_1 \cdot \bar{Y}_2 \cdot \bar{Y}_3 \cdot \bar{Y}_4 \cdot \bar{Y}_5 \cdot \bar{Y}_6 \cdot \bar{Y}_7$$

$$V = Y_7 + Y_6 + Y_5 + Y_4 + Y_3 + Y_2 + Y_1 + Y_0$$



The overbar terms are priority masks: a lower request contributes only when every higher request is inactive.

Logic Diagram (Block View)



Working example: the highest request wins

Trace two input patterns through the same encoder.

CASE 1 · CONFLICT

Y6 = 1 and Y2 = 1

Y6 outranks Y2, so the encoder selects Y6.

A2 A1 A0 = 110

V = 1. The Y2 request is ignored by the priority masks.

CASE 2 · SINGLE REQUEST

Y2 = 1 only

With no higher request active, Y2 is eligible to win.

A2 A1 A0 = 010

V = 1. The same output now identifies Y2 directly.

The selected code depends on the highest asserted input, not on how many inputs are active.

Verification and course-module connections

The encoder is a compact project that links specification, logic synthesis, storage, and system control.



Verification path: test the truth table, simplify with Boolean algebra or K-maps, then check the gate implementation against representative single-request and conflict cases.

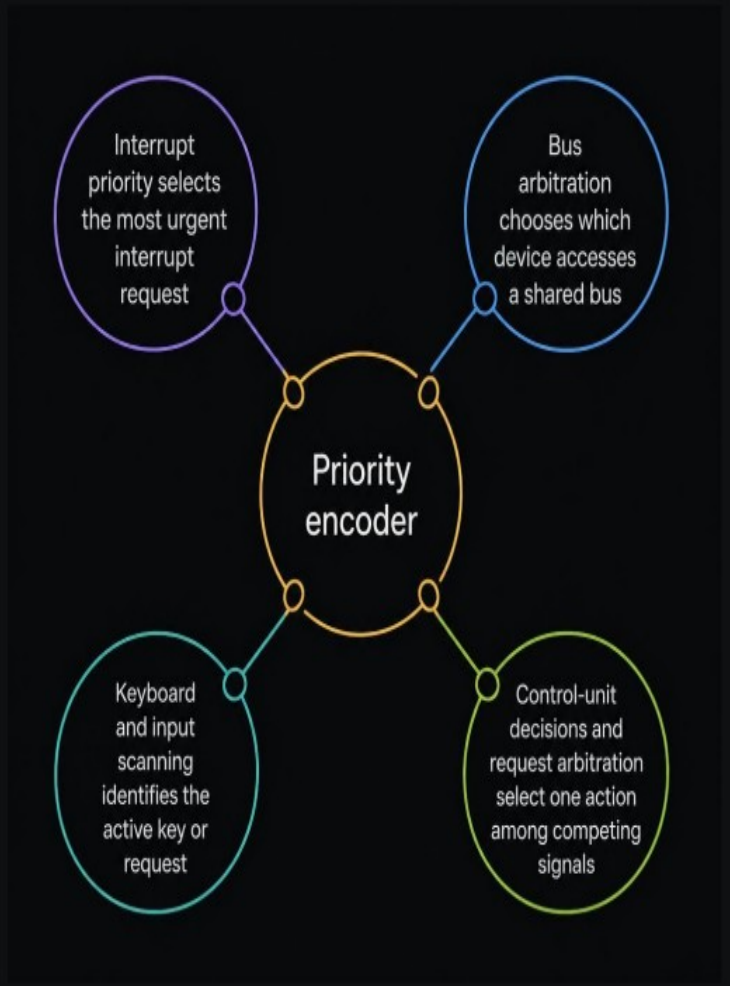
Applications and key takeaway

Priority logic turns many competing requests into one compact binary decision.

That decision can drive the next stage of a larger digital system, provided the valid flag accompanies the encoded address.

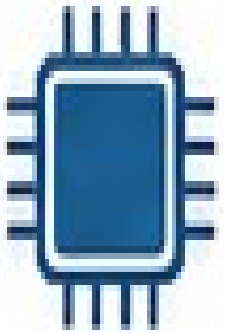
VIVA PROMPT

What changes if priority is reversed or the inputs are active-low?

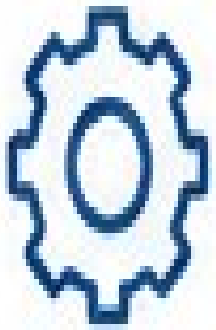


11. Real-World Applications

- Digital logic circuits (computers, calculators)
- Memory design (RAM, ROM)
- Microprocessors and controllers
- Communication systems
- Automation and control systems







13. Key Takeaways

1. Understand the basic Boolean operators and laws.
2. Construct truth tables for logical expressions.
3. Use algebraic simplification and K-Map method.
4. Draw logic circuits from simplified expressions.
5. Apply in real-world digital systems.

Conclusion

Boolean algebra is a powerful tool that simplifies logic, reduces complexity, and forms the foundation of modern digital electronics and computing.

Simple logic. Powerful applications.


thank
you